

The Stored Program Computer

In this section we examine one way to build a simple microprocessor. You all know that at the lowest level, the programming model for a microprocessor comprises some registers, memory for program and data, plus a set of instructions that is capable of manipulating these components.

The question is, how does it all work together? In other words, if you were to design a microprocessor from the ground up, how would it be done?

We'll explore this question by building our own microprocessor. The programmer's model for this processor is shown in Figure 1. It will be an 8-bit microprocessor with 512 bytes of byte-wide RAM. The RAM will store both program and data. There will be two 8-bit registers, A and B, and a condition-code register CC. Finally, two 8-bit input ports and two 8-bit outputs ports are provided for digital I/O.

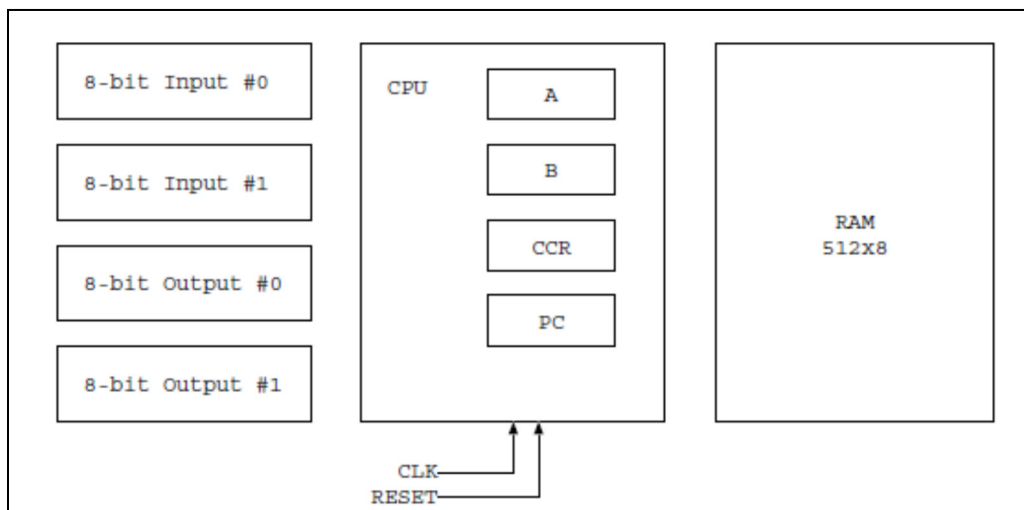


Figure 1: Programmer's model for an 8-bit microprocessor to be implemented in VHDL

We also need to decide upon an instruction set. Table 1 shows the instructions that we will implement for starters, just enough to be able to accomplish some useful things.

Our first design decision involves the underlying hardware architecture. Already, our choice of programmer's model implies certain constraints:

- Given that we have 512 addresses, this requires 9 bits of address data but we have an 8-bit architecture. We will therefore split our RAM into two "pages" or banks, page 0 for addresses 0-255 and page 1 for addresses 256-511.
- We will need an 8-bit data path to transfer data to/from the RAM and to/from the I/O registers, plus a 9-bit address path to the RAM and a 1-bit address path to the I/O registers.
- We have additional decisions to make that are not necessarily constrained, but we need to make choices nonetheless:
 - Upon reset, execution begins at address 0
 - The I/O input registers cannot be written and the I/O output registers cannot be read

Table 1: Microprocessor Instruction Set

Instruction	Encoding	Function	Description
LOAD M,R	000000pr mmmmmmmm	$R \leftarrow (M)$	Load register A ($r=0$) or B ($r=1$) with the contents of memory location M (in page p)
STOR R,M	000001pr mmmmmmmm	$(M) \leftarrow R$	Store register A ($r=0$) or B ($r=1$) to memory location M (in page p)
ADD R	1000000r	$R \leftarrow A+B$	Add registers A and B and store the result in either A ($r=0$) or B ($r=1$)
SUB R	1001000r	$R \leftarrow A-B$	Subtract register B from A and store the result in either A ($r=0$) or B ($r=1$)
LSL R	1010000r	$R \leftarrow R \ll 1$	Shift the contents of register A ($r=0$) or B ($r=1$) one bit to the left
LSR R	1011000r	$R \leftarrow R \gg 1$	Shift the contents of register A ($r=0$) or B ($r=1$) one bit to the right
XOR R	1100000r	$R \leftarrow A \oplus B$	Exclusive-OR registers A and B and store the result in register A ($r=0$) or B ($r=1$)
COM R	1101000r	$R \leftarrow \sim R$	Replace register A ($r=0$) or B ($r=1$) with its one's complement
NEG R	1110000r	$R \leftarrow -R$	Replace register A ($r=0$) or B ($r=1$) with its two's complement
CLR R	1111000r	$R \leftarrow 0$	Set register A ($r=0$) or B ($r=1$) to 0
OUT R,P	000010ir	$I \leftarrow R$	Write register A ($r=0$) or B ($r=1$) to output port #0 ($i=0$) or #1 ($i=1$)
IN P,R	000011ir	$R \leftarrow I$	Read from input port #0 ($i=0$) or #1 ($i=1$) and store in register A ($r=0$) or B ($r=1$)

1. Processor Architecture

With our data and control paths somewhat determined, let's take the programmer's model and think about what kind of implementation architecture is required. We will use the architecture shown in Figure 2, to be explained in the subsequent sections.

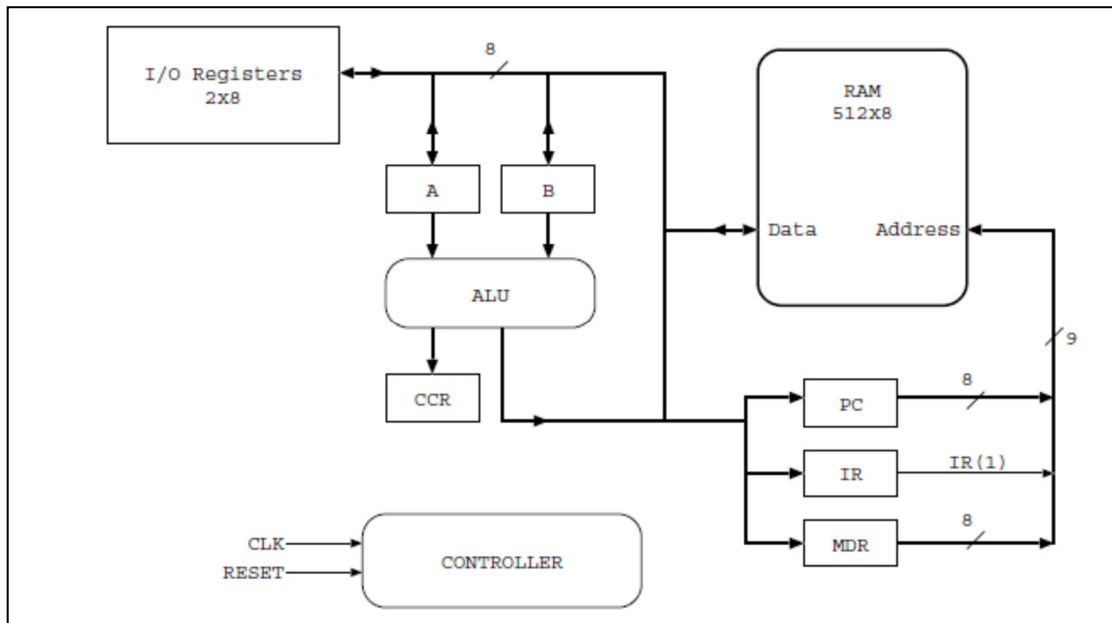


Figure 2: Internal architecture of 8-bit microprocessor

1.1 The Data Bus.

An 8-bit data bus runs throughout our microprocessor. During the implementation of the LOAD instruction, for example, we will use the bus to transfer data from the RAM to either the A or the B register. During the OUT instruction, we will use the data bus to transfer data from either the A or the B register to one of the I/O registers.

Note that the data bus is not a true bus in the sense that it has multiple output-enabled drivers. It is, in fact, implemented with multiplexers. For example, the A register may take on a new value driven by an I/O register, driven by the RAM, or driven by the ALU. In a traditional discrete-logic implementation of a controller, each of these three drivers would have output-enable drivers that would normally be tri-stated until it was their turn to “drive the bus”. This is desirable with respect to wiring as only 8 wires need to be connected between all devices connected to the data bus. However, performance suffers due to capacitive loading, leakage currents, and turn-on/turn-off times of the output-enabled drivers. Choosing the source of data for storage into the A register using a multiplexer requires additional combinational logic and more “wires” but does not require output-enable drivers.

1.2 The Registers.

The A, B, and CCR registers will simply be 8-bit registers connected to the main data bus and clocked by control signals from the Controller (to be discussed later). The CCR register will comprise at least 3 bits:

- The Z bit is set when the result of an ALU operation is 0
- The V bit is set when the result of an ALU operation has overflowed (signed two’s complement overflow)
- The N bit is set when the result of an ALU operation is negative

We will assume that all arithmetic registers (A and B) and ALU operations implement signed, two’s complement arithmetic.

1.3 The PC, IR, and MDR.

We will need to implement 3 registers to help us execute instructions. These are not seen in the programmer’s model (except for the PC) as they work behind the scenes. After we examine the execution cycle operation below, the usage of these registers will become more clear.

The PC is the Program Counter. It is a 9-bit register storing the address of the next instruction to execute. Upon reset, the PC is initialized to 0. After executing the first instruction, the PC will be either 1 or 2, depending upon whether that instruction was a 1-byte or 2-byte instruction.

The IR is the instruction register. It will store the instruction currently being executed.

The MDR is the Memory Data Register. It will be used to store a temporary operand or data value that was read from memory.

1.4 The ALU.

The Arithmetic Logic Unit (ALU) is a combinational logic block where the arithmetic instructions (ADD, SUB, XOR, etc.) will be implemented.

1.5 The I/O Registers.

The four I/O registers (two inputs, two outputs) will be modeled as a single 2-byte RAM, where writing to the RAM implicitly writes to an output register, and reading from the RAM implicitly reads an input register.

1.6 The Controller.

This is the brains of the whole operation. It is responsible for sequencing the signals that control the flow of execution. The best way to explain the controller is to begin investigating how the above architecture can be used to implement our instruction set.

2. Execution Model – An Example

Before describing the general execution model, let's take a look at some concrete examples of instruction implementations. How would we, for example, implement the instruction "**LOAD 123, A**", that is, load the contents of memory location 123 into register A.

Here are the steps:

- a. First we have to fetch the instruction to execute. The PC stores the address of this instruction. So....we have to place the 9-bit contents of the PC onto the address bus.
- b. Now we can read the contents of RAM at this address. This will be the byte representing the instruction to execute (known as the operation code, or *opcode*). We will place this opcode in the IR for later use. At this point, we also increment the PC to point to the next address.
- c. We see that the IR contains a LOAD instruction. The next thing we need is the byte following the instruction (the one where the PC is pointing at the moment) because that gives us the memory location that we need to read. So...once again we place the 9-bit contents of the PC onto the address bus.
- d. We once again read the contents of RAM at this address and store the operand in the MDR this time. Once again, we increment the PC.
- e. We then place the contents of the MDR onto the lower 8 bits of the address bus, and place bit 1 of the IR (the 'p' bit in the instruction) onto the MSB of the address bus to form a 9-bit address.
- f. We read the contents of RAM and place the result in either the A or B register.

The purpose of the controller, as you see, is to determine at any given time:

- who should be the “driver” of the data bus (really...the settings of the data multiplexers)
- who should be the “driver” of the address bus (again, a multiplexer in practice)
- which register/component should latch the contents of the data bus
- when should the PC be incremented

To get a better handle on how this microprocessor is going to implement all of our instructions, we need a more compact method of describing instruction execution. Writing out lots of words describing each instruction gets tedious and doesn’t allow us to discover patterns.

3. Register Transfer Level Descriptions

A more compact method of describing instruction execution is using Register Transfer Level (or RTL) descriptions. We already use RTL to describe overall instruction effect (column 3 of Table 1); now we use RTL to describe instruction execution at a lower level, referring specifically to architectural components.

For example, the 6 steps described above that implemented the “**LOAD 123, A**” instruction can be described in RTL as:

$IR \leftarrow MEM(PC),$	$PC \leftarrow PC + 1$	- get opcode
$MDR \leftarrow MEM(PC),$	$PC \leftarrow PC + 1$	- get operand
$ADDR \leftarrow IR(p) \& MDR$		- set memory address
$A \leftarrow MEM(ADDR)$		- read memory and store

Much more concise. The first line indicates that memory is being read at the address stored in the PC register, stored in the IR, then the PC register is incremented. The second line says that memory is being read at the address stored in the PC register, stored in the MDR, then the PC register is incremented. The third line says that a 9-bit address is constructed from the ‘p’ bit of the IR (bit #1) and the MDR. The final line assigns the contents of memory at the given address to register A.

Constructing RTL descriptions for our instruction set is a very useful stage in our microprocessor design development. We have introduced the RTL “language” by example, and you can see that, just like pseudo code, it is somewhat *ad hoc*. That’s fine because it’s just an intermediate step, a tool to take us from instruction set description to VHDL code.

An important aspect of RTL, however, is that it should map to the underlying capabilities of the hardware. For example, we could have rewritten the four lines of RTL above as:

$IR \leftarrow MEM(PC),$	$PC \leftarrow PC + 1$	- get opcode
$MDR \leftarrow MEM(PC),$	$PC \leftarrow PC + 1$	- get operand
$A \leftarrow MEM(IR(p) \& MDR)$		- read memory and store

Why is it necessary to have an extra step in which the address bus is formed? It seems wasteful, and the above 3 lines of RTL makes perfect sense.

Stay tuned. The short answer is that you have to be mindful of the capabilities of your underlying hardware and timing.

We will now introduce an ad hoc description for the ALU. We will describe the ALU in RTL as being a 3-parameter function:

Table 2: RTL Descriptions for the Instruction Set

Instruction	Instruction Fetch	Operand Fetch	Memory Access	Execute
LOAD M,R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$	$\text{MDR} \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$	$\text{ADDR} \leftarrow \{\text{IR}[\text{p}], \text{MDR}\}$	$R \leftarrow \text{MEM}(\text{ADDR})$
STOR R,M	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$	$\text{MDR} \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$	$\text{ADDR} \leftarrow \{\text{IR}[\text{p}], \text{MDR}\}$	$\text{MEM}(\text{ADDR}) \leftarrow R$
ADD R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(A, B, +)$
SUB R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(A, B, -)$
LSL R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(R, 0, < <)$
LSR R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(R, 0, > >)$
XOR R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(A, B, \oplus)$
COM R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(R, 0, \sim)$
NEG R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(0, R, -)$
CLR R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow \text{ALU}(0, 0, +)$
OUT R,P	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$I \leftarrow R$
IN P,R	$IR \leftarrow \text{MEM}(\text{PC})$ $\text{PC} \leftarrow \text{PC} + 1$			$R \leftarrow I$

ALU (x, y, f)

where x and y are two 8-bit inputs to the ALU and f is the function to compute (addition, subtraction, XOR, etc.). The ALU “function” returns a single 8-bit quantity.

For example, here is the RTL description for the execution of the “**ADD B**” instruction:

$IR \leftarrow \text{MEM}(\text{PC}),$	$\text{PC} \leftarrow \text{PC} + 1$	- fetch instruction
$B \leftarrow \text{ALU}(A, B, +)$		- execute it

This ALU function also implicitly updates the CCR register bits with the result of the operation.

Without further ado, let's write the RTL for our entire instruction set. Table 2 shows the RTL descriptions broken down into 4 possible phases: instruction fetch, operand fetch, memory access, execution. That we can implement any instruction with this 4-phase model is significant: it means that our controller component can have a very regular implementation.

As we can see, each instruction comprises either two or four RTL steps. In any case, the first RTL step for each instruction must be an operand fetch. When memory accesses are involved, an extra two steps must be taken to fetch the memory address (as part of the instruction), then prepare for the memory access, either read or write.

4. Controller State Machine

Since the RTL implementation is very regular for our instruction set, we can probably describe it as a state machine. Each one of the states will correspond to one of the above 4 phases. Figure 3 shows a possible state transition diagram for the controller.

Every instruction begins with an opcode fetch. A signal **Is4Phase** is then asserted (based upon the opcode) if the opcode is a 4-phase instruction, and negated if it is a 2-phase instruction.

There are lots of signals missing from the above state machine, but that is intentional. Even for a simple microprocessor such as ours, the number of signals would lead to a lot of clutter. That is not to say they aren't important, but I think it's useful to skip ahead to the timing diagrams and the VHDL code. We'll see that we don't really need to finish the above state transition diagram...VHDL code is sufficiently high level to finish the job.

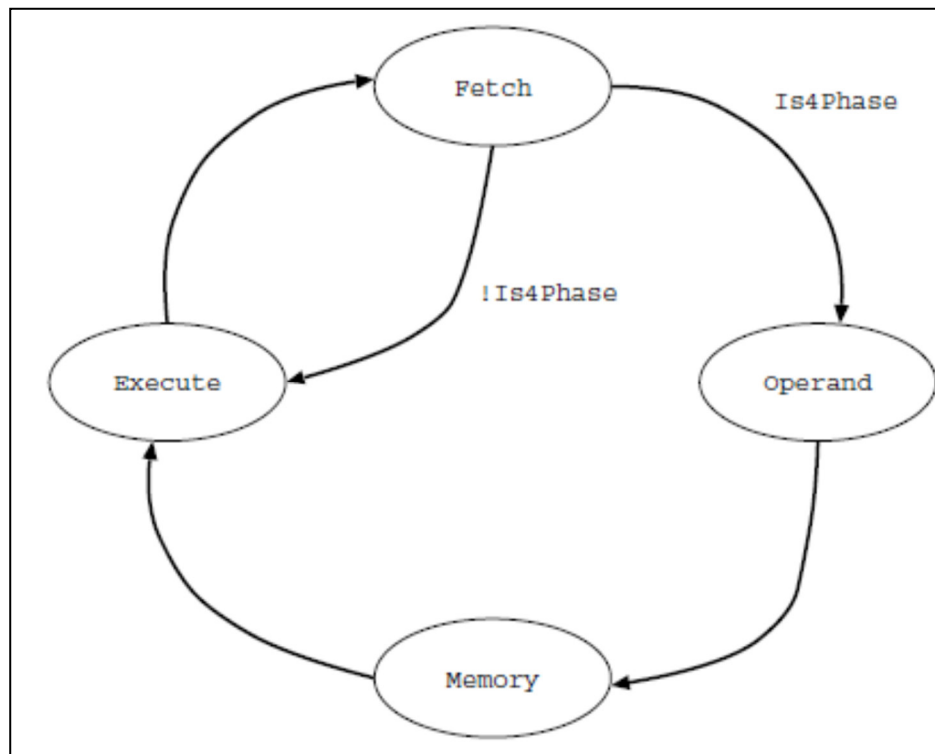


Figure 3: State transition diagram for the microprocessor controller. There are two paths through the state machine depending upon whether an instruction sequence requires two or four RTL phases.

5. Timing Diagrams

The majority of our microprocessor architecture simply represents registers and combinational logic (embedded in the controller state machine). Apart from setup times, hold times, and propagation delays for the registers and combinational logic, there aren't any constraints or operational details to worry about.

Not so for the memory. Here we have to worry about how the signals to the memory are sequenced and what its constraints are. To do this, we will look at the implementation of the memory and the timing diagrams for both the 2-phase and 4-phase execution cycles.

The memory we will use is modeled in Figure 4 above and implemented as a IP Core-generated Block Memory Single-Port RAM. There is a 9-bit address input, an 8-bit data input for memory writes, a separate 8-bit data output for memory reads, a clock, and a write enable. The interaction of these signals is summarized in the accompanying truth table.

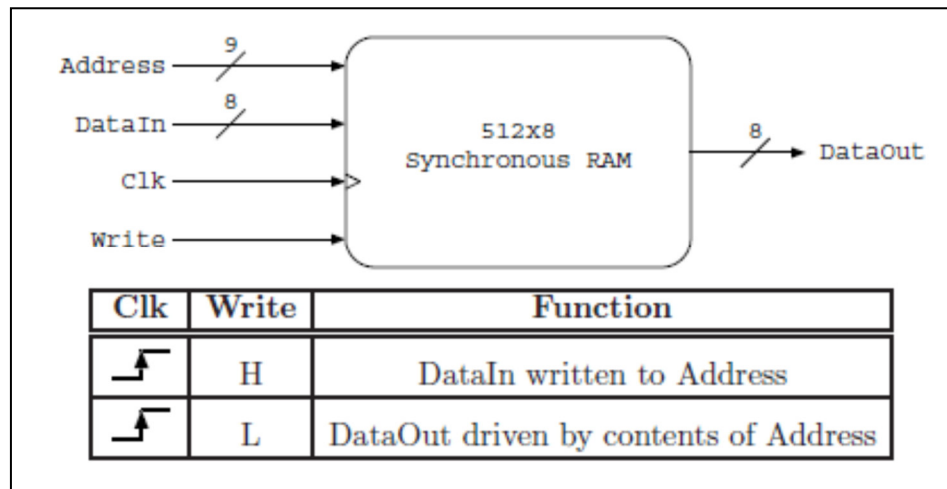


Figure 4: Functional model and truth table for the CPU memory component, a 512x8 synchronous RAM element.

The XC7S50 FPGA has 150 single-port on-chip RAM blocks, each capable of storing 18432 bits (or 18Kbits) or 75 dual-port on-chip RAM blocks, each capable of storing 36864 bits (or 36Kbits). The “port aspect ratio” (how many addresses by how many bits) is configurable so you can treat this ram as 32Kx1, 16Kx2, 8Kx4, 4Kx9, 2Kx18, 1Kx36 or 512x72. You can see that our 512x8 RAM easily fits within a single 4Kx9 RAM block, leaving the remaining 149 RAM blocks on the FPGA free for other things. Alternatively, we can give our microprocessor a larger memory; anything between 512x8 and 4Kx9 is “free” since it still uses only a single RAM block. However, we would have to go back to our architecture design and figure out how to widen our address bus – note that with the instruction set as described above, there is no way to reference any address above 511.

The RAM is synchronous such that reads and writes only occur on clock edges. For a write to the RAM, the address, data, and write enable signal have to be set up prior to a rising edge on **the clock**, at which point the write occurs. The write actually occurs after some propagation and processing delay but we'll just concern ourselves with functional behavior for now. Reads are simpler. When the write enable signal is low, a rising edge on the clock drives the **DataOut** bus with the contents of memory at the address latched in on the clock rising edge. Again, there will be some realistic clock-to-output delay.

Note that the **DataOut** bus is always driven; it is never tri-stated.

With the above in mind, let us examine the timing diagram for a 2-phase instruction cycle, Fetch and Execute. This diagram is shown in Figure 5 below. Note that we have both the regular clock CLK and we also construct its complement $\overline{\text{CLK}}$. This construction may be useful later on.

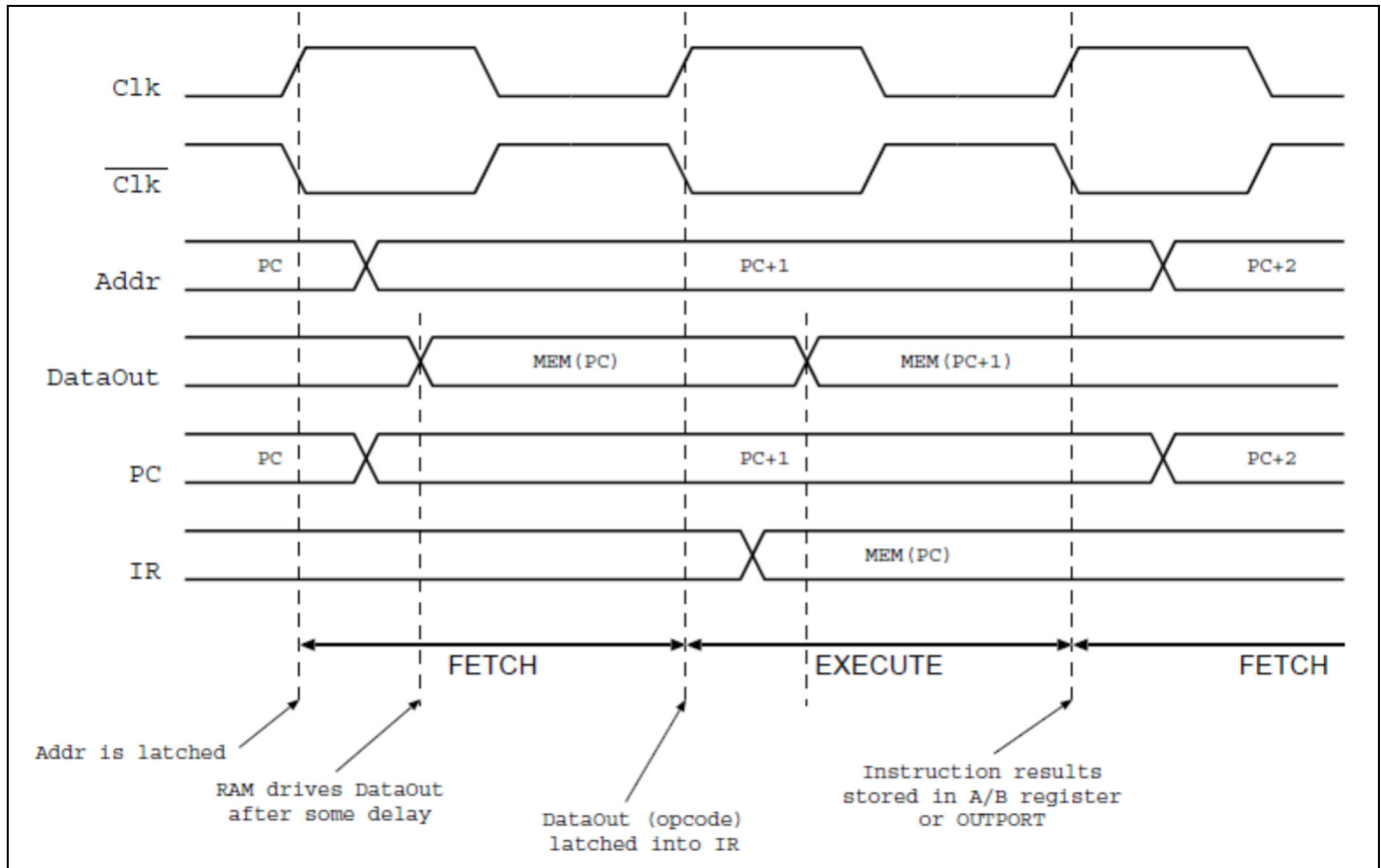


Figure 5: Timing diagram for a 2-phase instruction cycle. Not that the RAM write-enable signal is low throughout this entire diagram as only RAM read cycles are occurring.

In the first cycle, the current contents of the PC are enabled onto the address bus prior to the start of the cycle (the rising edge of CLK), at which time the address is latched into the RAM. Soon thereafter the RAM drives the contents of this address onto the data bus. The PC is incremented on the rising edge of CLK.

At the start of the Execute phase, the data bus is latched into the IR, thus storing the opcode. During the Execute phase, whatever work needs to be done for the instruction is done and the results placed on the data bus. At the start of the next Fetch phase, the data bus is latched into whichever register is to store the data (A, B) or perhaps latched into one of the I/O ports if an OUT instruction is being executed.

As can be expected, things are a wee bit more complicated for the 4-phase instructions. The timing diagram for an instruction that reads memory (e.g., LOAD) is shown in Figure 6 below.

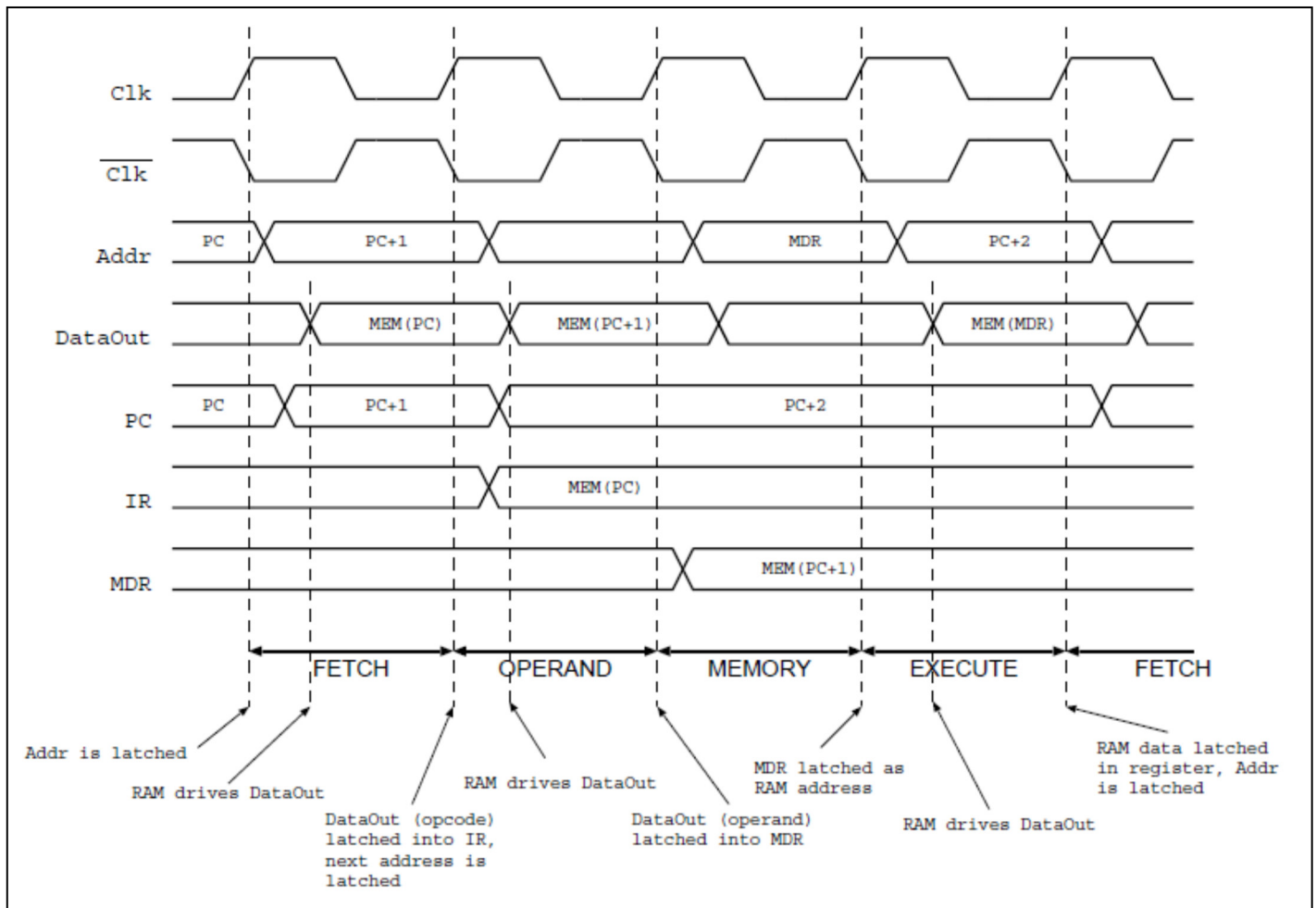


Figure 6: Timing diagram for a 4-phase instruction cycle (memory read). The RAM write-enable signal is low throughout this entire diagram.

The Fetch phase is the same as for the 2-phase cycle. The Operand phase is essentially just another Fetch phase, except the **DataOut** bus is now latched into MDR instead of IR at the beginning of the next phase. During the Memory phase, the MDR value is now placed on the address bus and is latched into the RAM at the start of the next phase. The Memory phase is the only time that the RAM's address bus is driven by the MDR instead of the PC. In the final Execute phase, the RAM is read and latched into its destination (e.g., register A) at the start of the next Fetch phase.

A memory write instruction (e.g., STOR) is the same except the data to be written to RAM is enabled onto the **DataIn** bus during the Memory phase and is latched into the RAM at the start of the Execute phase.

6. State Machine Implementation

With a better feel for what's going on, let's start taking a look at the synchronous component of the controller state machine. We have the states themselves, plus three synchronous registers: PC, IR, and MDR. Recall that the overall structure of next-state logic in VHDL is:

```
if(rising_edge(clk)) then
  case CurrState is
    when StateX =>
      ....
    when StateY =>
      ....
  end case;
end if;
```

Let's think about each of the above **CASE statements**, i.e., what needs to happen synchronously as we transition from one state to the next.

An important point to keep in mind is that each **CASE** statement indicates what should happen as we exit from the given state.

6.1 Fetch-to-Execute / Fetch-to-Operand.

During the Fetch phase, the opcode is enabled onto the data bus. At the start of the next phase, then, we need to latch this into the IR. We also need to determine whether our next state should be Operand (for 4-phase instructions) or Execute (for 2-phase instructions). We will write a function called **Is4Phase** that examines the data bus (where the opcode is currently being driven) and returns TRUE if this is a 4-phase instruction.

If transitioning to the Operand phase, we need to first increment the PC since this is a two-byte instruction. Thus, here is the VHDL code for the transition out of the Fetch phase:

```
when Fetch =>                                -- FETCH STATE
  IR <= Data;
  if(Is4Phase(Data)) then
    PC <= PC + 1;
    CurrState <= Operand;
  else
    CurrState <= Execute;
  end if;
```

Note that the assignment "**IR <= DATA**" implies that the data bus will be latched into the IR register on the rising edge that causes a transition *out* of the Fetch state. It is tempting to interpret the above as saying that the IR register should store the data bus while in the Fetch state, but remember that we are inside a clocked process: things only change on rising clock edges.

6.2 Operand-to-Memory.

At the end of the Operand phase, the operand is being driven onto the data bus and needs to be latched into the MDR:

```
when Operand =>                                -- OPERAND STATE
    MDR <= Data;
    CurrState <= Memory;
```

6.3 Memory-to-Execute.

Neither the MDR nor IR nor PC need to change. This one's a lay-up:

```
when Memory =>                                -- MEMORY STATE
    CurrState <= Execute;
```

6.4 Execute-to-Fetch.

This is probably the trickiest phase as we have to store the results of instruction execution somewhere. All of the CPU registers (A, B, I/O ports, etc.) are synchronous and need to be latched at the rising edge of **CLK**, which also starts the next Fetch cycle. Thus, this state transition may need to latch the data bus into some register, depending on the instruction. We will define several signals that indicate whether a register should be latched with data at this time:

```
-- Declare signals that indicate which registers are to be written from the
-- DATA BUS at the start of the next FETCH cycle
signal Exc_RegWrite : STD_LOGIC;    -- Latch DATA BUS in A or B
signal Exc_CCWrite  : STD_LOGIC;    -- Latch ALU status bits in CCR
signal Exc_IOWrite  : STD_LOGIC;    -- Latch DATA BUS in I/O
```

We only need one signal to indicate that register A or B is being written to since all of our instructions use bit 0 of the opcode to indicate the register to be written. Thus, IR(0) tells us where to write:

```
if(Exc_RegWrite = '1') then                -- Writing Result to A or B
    if(IR(0) = '0') then
        A <= Data;
    else
        B <= Data;
    end if;
end if;
```

Similarly, we always use bit #1 of the opcode to indicate which I/O register to write:

```
if(Exc_IOWrite = '1') then    -- Write to Outport0 or Outport1
  if(IR(1) = '0') then
    Outport0 <= DATA;
  else
    Outport1 <= DATA;
  end if;
end if;
```

Finally, we'll use 3 wires to represent the condition codes coming from the ALU. These will be latched into the CCR if required:

```
if(Exc_CCWrite = '1') then    -- Updating flag bits
  V <= ALU_V;
  N <= ALU_N;
  Z <= ALU_Z;
end if;
```

We also have to worry about the opcode fetch part by incrementing the PC and transitioning to the Fetch state. Here then, is an abbreviated version of the VHDL code for the Execute-to-Fetch state transition:

```
when Execute =>              -- EXECUTE STATE
  PC <= PC + 1;
  CurrState <= Fetch;
  if(Exc_RegWrite = '1') then
    -- see above
  if(Exc_IOWrite = '1') then
    -- see above
  if(Exc_CCWrite = '1') then
    -- see above
```

We're done with the synchronous and next-state logic for the state machine; now we have to think about the asynchronous components, i.e., the output logic. Let's first look at the address bus. It is driven differently depending upon the execution phase and the instruction.

In the Fetch and Execute phases, the address bus is always driven by the PC. During the Memory phase the address bus is driven by the MDR (plus the page select bit from the IR). Finally, during the Operand phase the address bus is not being used. We can encode this behavior into a single VHDL statement:

```
-- ----- Generate address bus -----
with CurrState select
  ADDR <= PC           when Fetch,
           PC           when Operand,    -- really a don't care
           IR(1) & MDR  when Memory,
           PC           when Execute,
           PC           when others;    -- just to be safe
```

Hey, that was pretty simple! Let's do the data bus driver now. This is going to be more complicated. During the Fetch and Operand phases, the RAM is always driving the bus. During the Memory phase, either the A or B register is driving the bus during memory writes (as these are the only things we can write to memory given our current instruction set). During memory read instructions, there is no driver of the data bus during the Memory phase so we may as well pretend it's a memory write (this will save some logic). Finally, during the Execute phase, the driver of the data bus totally depends upon the instruction. Let's do everything other than the Execute phase:

```
process(CurrState)
begin
  case(CurrState) is
    when Fetch | Operand =>
      DATA <= RAM_DATA_OUT;

    when Memory =>
      if(IR(0) = '0') then
        DATA <= A;
      else
        DATA <= B;
      end if;

    when Execute =>
      -- to be completed
  end case;
end process;
```

Notice that since this is not a clocked process, each **CASE** statement indicates what should happen when the FSM is in the given state, not on transitions from state to state. The first **CASE** statement, for example, indicates that the data bus should be driven by the RAM DataOut signal for the entire time that the CPU is in either the Fetch or Operand state. This assignment does not imply (as it did for the next-state clocked process) that the DATA bus is somehow latched on a rising edge.

7. ALU Implementation

The final big-picture piece of the puzzle is the purely combinational ALU. We see that it sometimes manipulates one operand (NEG, COM instructions for example) and sometimes two operands. There are eight different functions it is required to perform, and we have cleverly encoded the function to be performed in bits #6 through #4 of the opcode. Table 3 below summarizes how these bits encode ALU function.

Table 3: Encoding of ALU Functions

IR(6 downto 4)	ALU Function
000	ADD
001	SUB
010	LSL
011	LSR
100	XOR
101	COM
110	NEG
111	CLR

In VHDL, we have the concurrent signal assignment:

```
signal ALU_FUNC : STD_LOGIC_VECTOR(2 downto 0);
.....
ALU_FUNC <= IR(6 downto 4);
```

The ALU needs to return an 8-bit result plus the state of the three condition codes, N, V, and Z. We will implement the ALU as a VHDL entity, perhaps looking forward to reusing it in the future or implementing platform-specific optimizations depending upon which FPGA we are targeting.

The code for our ALU is shown in Figure 7. It is hopefully fairly self-explanatory. Some things to notice are the use of a function for computing the state of the V bit, and the use of the concatenation operator ‘&’ to simulate logical shifts and the occasional necessary type conversions required between SIGNED and STD_LOGIC_VECTOR.

Also note that the addition ‘+’, subtraction ‘-’, exclusive-or ‘XOR’, and complement ‘NOT’ operators are all being used in their overloaded forms, applied to the **SIGNED** type. This support is provided by the **ieee.std_logic_arith** package. Also note that two’s complementation (the ‘NEG’ function) is implemented as an integer-minus-**SIGNED** subtraction, another interesting overloading of the ‘-’ operator.

The computation of the N and Z bits is fairly simple: when any result is 0 the Z bit is set, when any result is negative (i.e., the MSB is set) then N is set. The V bit is set when two’s complement overflow occurs during addition or subtraction, or when we try to take the two’s complement (NEG operator) of -128 (since +128 isn’t representable in 8 bits). In all other cases, we set V to 0.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;

entity alu is
port(A,B : in SIGNED(7 downto 0);
      F : in STD_LOGIC_VECTOR(2 downto 0);
      Y : out SIGNED(7 downto 0);
      N,V,Z : out STD_LOGIC);
end alu;

architecture Behavioral of alu is
-- ---- Function to determine if overflow occurs ----
function ISOVERFLOW(constant A,B,R : signed(7 downto 0)) return STD_LOGIC is
variable RETVAL : STD_LOGIC;
begin
  if((A(7) and B(7) and not R(7)) OR (not A(7) and not B(7) and R(7))) = '1' then
    RETVAL := '1';
  else
    RETVAL := '0';
  end if;
  return RETVAL;
end function;
end architecture;
```

```

begin
-- ----- Update Condition code flag bits -----
process(A,B,F)
variable RETVAL : signed(7 downto 0);
begin
    V <= '0';
    if(F = "000") then      -- Addition
        RETVAL := A + B;
        V <= ISOVERFLOW(A,B,RETVAL);      -- OVERFLOW flag
    elsif(F = "001") then   -- Subtraction
        RETVAL := A - B;
        V <= ISOVERFLOW(A,-B,RETVAL);
    elsif(F = "010") then   -- LSL
        RETVAL := A(6 downto 0) & '0';
    elsif(F = "011") then   -- LSR
        RETVAL := '0' & A(7 downto 1);
    elsif(F = "100") then   -- XOR
        RETVAL := signed(STD_LOGIC_VECTOR(A) xor STD_LOGIC_VECTOR(B));
    elsif(F = "101") then   -- COM
        RETVAL := signed(not STD_LOGIC_VECTOR(A));
    elsif(F = "110") then   -- NEG
        RETVAL := 0 - A;
        if(STD_LOGIC_VECTOR(RETVAL) = "10000000") then
            V <= '1';
        end if;
    else                    -- CLR
        RETVAL := "00000000";
    end if;

    if(STD_LOGIC_VECTOR(RETVAL) = "00000000") then      -- ZERO flag
        Z <= '1';
    else
        Z <= '0';
    end if;

    if(RETVAL(7) = '1') then      -- NEG flag
        N <= '1';
    else
        N <= '0';
    end if;

    Y <= RETVAL;
end process;
end Behavioral;

```

Figure 7: VHDL implementation of the microprocessor ALU

8. Instruction Implementation

All of the pieces are in place, we are ready to actually start implementing instructions. We need only focus on the Execute phase of the instruction cycle, the “**CASE Execute**” clause of the procedural block manipulating the data bus (see page 14).

Clearly we need to take different actions depending upon the opcode stored in the IR register (actually, only the first 7 bits since bit 0 always encodes a register choice). Our general structure will be a **CASE** statement:

```
case IR(7 downto 1) is
  when "1000000" =>
    ....    -- ADD R instruction
  when "1001000" =>
    ....    -- SUB R instruction
end case;
```

What do we need to do, for example, to implement the ADD instruction? We need to:

- Pass the A and B registers to the ALU. The ALU function is already derived from bits 6 through 4 of the IR.
- Enable the ALU output to drive the data bus
- Set the **Exc_RegWrite** signal to latch the ALU result into a register

Here, then, is our VHDL implementation of the ADD instruction:

```
when "1000000" =>          -- ADD R
  ALU_A <= A;
  ALU_B <= B;
  DATA <= ALUOUT;
  Exc_RegWrite <= '1';
  Exc_CCWrite <= '1';
```

That's it! And guess what...the code for the SUB instruction is exactly the same, and it isn't much different for the remaining arithmetic instructions.

How about the OUT instruction now? Either A or B drives the data bus, depending upon IR bit 0. The data bus then will be latched into one of the output registers at the start of the next cycle.

```
when "0000100" | "0000101" =>      -- OUT R,P
  if (IR(0) = '0') then
    DATA <= A;
  else
    DATA <= B;
  end if;
  Exc_IOWrite <= '1';
```

Finally, let's look at a memory read instruction “**LOAD M, R**”.

```
when "0000000" | "0000001" =>          -- LOAD M.R
    DATA <= RAM_DATA_OUT;
    Exc_RegWrite <= '1';
```

And we have ourselves a microprocessor. A few details have been left out, such as the actual declaration of the signals involved (such as RAM_DATA_OUT) but these are nothing more than “wires”. The full source code to this microprocessor has been made available to you for study.

9. Postmortem

This is about as simple a microprocessor as you can imagine, yet it already has a certain level of complexity. Imagine adding in interrupts, subroutines, branches, multi-byte registers and memory accesses...the complexity goes way up. The underlying structure of our design, however, is the basis for much more complex designs.

Some texts describe examples of more complicated microprocessors. These texts comprise thousands of lines of VHDL code, whereas our own simple microprocessor has few hundred lines of code. If you study the code for the more complicated processors, you will recognize many components of the structure we have presented: a state machine for sequencing phases of execution, an ALU, several internal registers, etc.

10. But Why?

Given that there are an overwhelming number of choices of microprocessors already in existence, why would we worry about designing another one in VHDL?

Well...firstly, as a chip designer, you may be given the task of designing yet another choice in the overwhelming number of choices.

Second, recall the SoPC architectures presented earlier. One of their strengths was an integrated processor core. A new microprocessor design may be just one component of a specialized chip that is designed to integrate a lot of functionality in a very small area. Altera's Nios CPU and Xilinx's MicroBlaze and PicoBlaze CPU's are embeddable processors that can be incorporated into a VHDL design and hooked up to your own custom logic, or even duplicated to implement a parallel processing system.

Third, there is the issue of performance. We may not be happy that a PIC microcontroller can only execute at an internal clock frequency of 5 MHz. A custom microcontroller core may give us faster performance, plus allow us to implement just the right instructions for the job. Digital signal processing (DSP) specialized processors include just such instructions, such as a simultaneous multiply and add instruction. Imagine, for example, taking a simple, off-the-shelf core like that of the 8051 (available for free at <http://www.opencores.org>) and adding in your own specialized instructions and additional hardware to drive a VGA display. That would be a totally different way to implement your Project #2. The 8051 architecture has lots of free assemblers, compilers, etc. available for it so you could hit the ground running with this customized “soft core”.

Finally, forget microprocessors. The paradigm of having a state machine sequence RTL operations can be used for very specific designs, not just general-purpose computing. Many peripherals, such as UART's, are really specialized microprocessors in disguise.